基于Flink的实时视频分析系统设计与实现

刘应亮，李文生
（北京邮电大学计算机学院，北京 100876）

摘要：随着智能视频监控的快速发展，对监控视频数据进行实时视频分析的需求日益增多，由于视频分析算法的复杂度相对较高，对视频数据进行图像处理时比较耗时，导致实时分析出的结果延迟较高，且分析后的目标视频帧率较低，无法保证视频的流畅度。因此，本文设计了基于流式计算框架Flink的实时视频分析流水线，并在此基础上实现了实时视频分析系统。系统的测试结果表明该系统对监控视频调用实时视频分析算法进行计算后，能够降低分析结果的整体延迟和帧率，从而满足实时性和流畅度要求。

关键词：计算机应用技术；Flink；视频分析；流式计算
中图分类号：TP39

Design and implementation of real-time video analysis system based on Flink

Liu Yingliang, Li Wensheng
(School of Computer Science, Beijing University of Posts and Telecommunications, Beijing 100876)

Abstract: With the rapid development of intelligent video surveillance, the demand for real-time video analysis of surveillance video data is increasing. Due to the relatively high complexity of video analysis algorithms, it takes time to process video data, leading to high latency of real-time analysis results. And the frame rate of the target video after analysis is low, which cannot guarantee the fluency of the video. Therefore, this paper designs a real-time video analysis pipeline based on the streaming computing framework Flink, and implements a real-time video analysis system based on this. The test results of the system show that the system can reduce the overall delay and frame rate of the analysis results after calculating the real-time video analysis algorithm of the surveillance video, so as to meet the real-time and fluency requirements.

Key words: computer application; Flink; video analysis; streaming computing

0 引言

近年来，网络传输、视频编码压缩、计算机视觉等技术都取得了重大突破，智能监控系统[1]也因此得到迅猛的发展。智能视频监控系统在传统视频监控系统之上，引入了各种智能算法，通过对视频图像进行分析，能够实现异常事件检测、入侵检测等功能，自动发送报警信息。相比传统的监控系统，智能监控系统可以提高监控效率，从而减轻监控人员的工作压力。

随着监控视频数据的迅猛增长，采用传统方式对视频进行分析处理的缺点开始显现，效率不足，困难重重。而近些年兴起的大数据技术可以为视频数据的处理提供新的思路和架构，使得人们得以更智能、高效地处理和分析海量的视频监控数据。这也给视频监控系统在高可用、低延迟上提出了更高的要求。由于视频图像这种非结构化数据处理较为复杂，导致图像处理算法的处理时间相对较长[2]。如果要实现视频分析的实时性，需要在图像分析算法、硬

作者简介：刘应亮（1996-），男，硕士在读，主要研究方向：多媒体技术
通信联系人：李文生（1966-），女，副教授、硕导，主要研究方向：数据库系统及应用、网络信息处理、多媒体技术. E-mail: wenshli@bupt.edu.cn

件设施或者处理架构上进行改进。

因此，本文设计了基于流式计算框架 Flink[3]的实时视频分析系统，该系统结合了流媒体技术、视频编解码技术以及 Vue[4]前端技术，设计实现了基于 Flink 的实时视频分析流水线方案。系统可以对采集到的监控视频进行实时的视频分析并展示分析结果，能够很好地降低延迟，并保证视频的流畅度。

1 系统整体设计

本文设计实现的实时视频分析系统的结构层次分为采集层、处理层、存储层、展示层。系统的整体架构如图 1 所示：

图 1 系统整体架构

Fig. 1 system architecture

其中，采集层负责采集各个摄像头的视频流数据采集，包括监控设备的注册登录，以及对应 RTSP[5]视频流的获取。处理层负责对采集到的视频流数据进行解码，并调用视频分析算法的接口处理解析到的视频帧，在该层次中需要对视频流解码出的视频帧序列逐帧或隔帧进行图像处理，是一种流式的处理过程，因此本文设计实现了基于 Flink 的实时视频分析流水线。存储层主要负责存储处理层中视频分析的结果，对结果进行一些持久化的操作从而方便后续的查询。展示层负责将视频分析的结果传输给用户并在前端页面进行实时的展示，提供可视化的操作。

2 基于 Flink 的实时视频分析流水线设计

视频分析功能，是智能视频监控系统中“智能”的重要体现，为了将视频分析算法和平台解耦，需要将算法从传统的本地调用模式转变为服务化的模式，通过 HTTP 请求或 RPC 的方式调用算法的 API。调用算法服务一般是视频分析中耗时最长的部分，如何通过软件层面的方法对实时视频分析的延迟和帧率进行优化，是本文的重要研究内容。

本文在对实时视频分析的方法进行分析研究后，设计并实现了基于 Flink 的实时视频分析流水线。在异步调用算法的处理过程中，假设处理每帧数据的平均响应时间为$\overline{T}$，异步处理的并发量为P，那么为使得帧与帧之间的平均间隔s'小于等于给定的值s，需要满足：$s'=\frac{\overline{T}}{P}\le s$，而因为每帧处理的响应时间不固定，存在$T_i>T_{i-1}$，可能导致视频卡顿，需要设置异步处理的超时时间$t=\overline{T}$，使得处理超时的帧直接返回原始帧，以保证视频的流畅度。

但是使用异步的方式会导致处理数据的乱序，因此本文使用水位线和时间窗口机制解决异步带来的乱序问题。记录每个事件（每帧数据）的事件时间戳，定期生成水位线，水位线为一个时间戳，其算法为$t_{wm}=t_{max}-t_{delay}$，其中 t_{max} 为当前事件的最大时间戳，t_{delay} 为允许的最大延迟，该水位线表示t_{wm} 之前的数据都已到达，当满足时间窗口的触发条件时，会触发窗口函数，对窗口内的数据按照事件时间戳进行排序。对于超过最大延迟时间还未到达的数据，默认丢弃。如图 2 所示为水位线机制示意图。

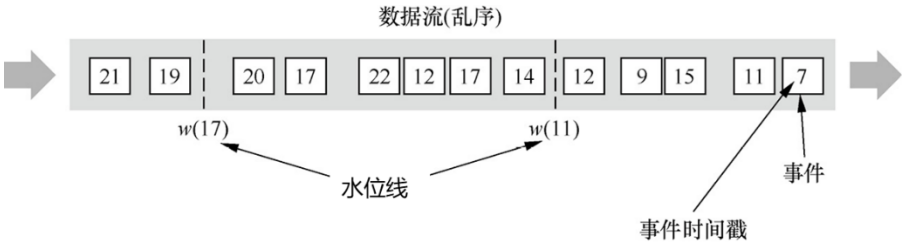

图 2 水位线机制

Fig. 2 watermark

在此基础上，结合 Flink 的编程模型，将整体流水线设计如下图 3 所示：

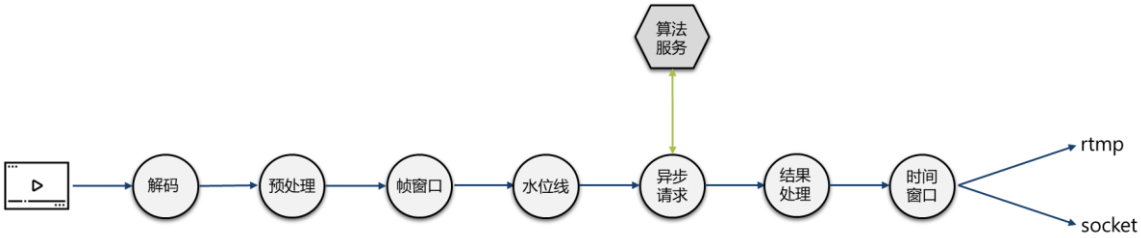

图 3 基于 Flink 的实时视频分析流水线

Fig. 3 pipeline design

对于有帧间依赖的算法服务，本文设计了帧窗口，方便对一个窗口内的所有帧统一进行处理；根据不同的算法类型，数据下沉的方式也不同，对于需要实时绘制 Bounding Box 并展示的，将帧编码为 RTMP[6]格式的视频流供客户端播放；对于需要实时发送通知提醒的，将算法返回的结果通过 WebSocket 发送给客户端。

在该流水线下，视频的延迟和帧率（FPS）计算如下：

$$T_{delay} = T_d + T_b + T_p + T_a + T_t + T_{wc} + T_{wt} + T_e$$

$$F_{out} = \frac{1000}{\frac{T_b}{P_b} + \frac{T_p}{\sum_1^n P_i} + \frac{T_a}{P_a}}$$

其中的符号说明如下表 1 所示：

表 1　符号说明

Tab. 1 symbol description

符号	符号含义
T_{delay}	处理后的视频的整体延迟
T_d	视频解码耗时
T_b	图片预处理耗时，如将图片帧转为 byte 数组或 Base64 等格式的数据
T_p	算法服务响应耗时
T_e	视频编码耗时
T_a	结果处理耗时，如图片格式转换、绘制 Bounding Box 等操作
T_t	网络传输耗时
T_{wc}	在对一段帧序列统一处理时引入帧窗口产生的延时
T_{wt}	防止帧乱序而引入时间窗口产生的延时，即时间窗口的大小
T_{wm}	水位线的最大允许延迟
P_i	第 i 个算法服务实例支持的并发数量
P_b	预处理过程的并发数量
P_a	结果处理过程的并发数量
F_{out}	输出视频流的 FPS

若要保证输出视频至少达到流畅的要求，需要 $F_{out} \geq 25$，即 $\frac{T_b}{P_b} + \frac{T_p}{\sum_1^n P_i} + \frac{T_a}{P_a} \leq 40$，实际处理中为异步调用设置的并行度以此作为依据。对于不需要实时绘制算法处理结果的，没有 FPS 限制，可适当降低并行度。

3　系统实现及测试

3.1　系统实现

整个系统的开发采用前后端分离的模式，即客户端和服务端分离，其中服务端使用 Spring Boot[7]技术，实现系统主要的功能逻辑，并为客户端提供接口服务。服务端中各个模块的实现如下：

采集层：首先需要对指定的视频监控设备进行登录验证。登录成功后，对于支持 RTSP 协议的设备，设备作为推流端直接向外推送 RTSP 视频流，服务端进行收流。对于无法支持 RTSP 协议的设备，通过调用设备厂商提供的 SDK 接口获取视频数据。以海康威视的网络摄像头为例，使用海康威视的官方 SDK 实现监控设备的访问，海康 SDK 是使用 C++开发的一组程序，而本文实现的系统是用 Java 开发的，因此需要使用 JNA（Java Native Access）技术进行访问。JNA 技术基于 JNI（Java Native Interface），比 JNI 封装了更高级的 API。

处理层：基于 Flink 的编程模型和视频分析算法服务，本文设计了基于 Flink 的实时视

频分析流水线。对监控视频进行实时分析时，作为参数，首先指定监控设备和将调用的视频分析算法，然后调用对外暴露的实时视频分析接口，该接口将提交并执行 flink 任务。 在基于 Flink 的实时视频分析流水线中，整个数据的流向如下：首先以该设备的 RTSP 视频流作为输入数据 Source，解码成连续的图像帧序列，预处理的过程主要是为了适应视频分析算法服务的接口参数，根据需要调整转换图像的尺寸和格式。预处理后的数据流根据需要进行数量窗口（CountWindow）的划分，之后为该数据流指派事件时间戳和水位线生成机制。接着数据流调用异步函数（AsyncFunction），在该函数中，通过 RestTemplate 发送 Http 请求，调用外部的图像处理算法服务接口，并将返回的结果放入数据流中。因为返回的结果一般为 Bounding Box 或者日志记录，所以在结果处理步骤中，需要根据返回结果的不同类型将上一步的结果进行进一步的处理，如将 Bounding Box 绘制在对应的图像帧上。处理后的数据流经过时间窗口的处理（TimeWindow）以保证数据的顺序。最后数据根据不同的需求输出到不同的 Sink 中。

存储层：采用分布式文件系统 HDFS 存储处理层中视频分析的结果，目的是方便后续用户的查询，利用 Flink 自带的连接器 Hadoop FileSystem Connector 将 HDFS 作为 Flink 数据下沉的 Sink。

展示层：在处理层中实时视频分析输出的结果，要在客户端实时地展示出来。对于需要实时绘制 Bounding Box 的类型，将结果编码为 RTMP 格式的视频流，然后经由 RTMP 流媒体服务器推送到客户端。对于日志记录类型，将算法返回的结果通过 WebSocket 发送给客户端。

客户端使用 Vue 进行开发，同时使用 Axios 作为向服务端发送 HTTP 请求的交互工具。

3.2 性能测试

本文进行了不同处理耗时、并发数量下的性能测试，其中处理耗时指的是异步处理过程中调用外部视频分析算法所耗费的时间。实验结果表明，保持异步调用的平均处理间隔在 30ms 以内，即可保证视频帧率达到至少 25fps，满足流畅性的要求。如表 2 所示为性能测试的实验结果，其中的最后一行数据为调用人脸检测算法服务的测试结果，在平均耗时 170ms 的情况下也能保证输出视频的帧率在正常范围。

表 2 实时视频分析性能测试

Tab. 2 performance test

处理耗时/ms	并发数量/个	帧间隔/ms	帧率/fps	延迟/ms
100	5	38.77	25	170.271
100	10	38.701	25	144.916
200	10	38.87	25	261.14
300	10	40.676	24	393.157
400	10	57.346	17	514.23
400	15	38.678	25	465.93
500	20	38.681	25	536.908
170	8	38.74	25	257.10

本系统也对比了 Microsoft Cognitive Services 的开源项目，该项目使用生产者消费者模式对实时视频流中提取的帧执行近实时分析，使用的算法服务为 Microsoft Azure

提供的 Face API，视频源为本地摄像头视频。在相同服务器环境、视频源、算法服务、和检测频率的条件下，二者的性能对比如表 3 所示，结果表明使用本文提出的方法能够有效降低延迟，提高视频帧率。

表 3　实时视频分析性能对比

Tab. 3　performance comparation

方法	延迟/ms	帧间隔/ms	帧率/fps
本系统	784.10	38.42	26
Microsoft Cognitive Services	2443.83	50.10	20

4 结论

本文分析研究了流式计算框架在实时视频分析中的重要性，设计了基于 Flink 的实时视频分析流水线，并结合流媒体技术、视频编解码技术以及 Vue 前端技术，设计实现了基于 Flink 的实时视频分析系统。系统可以对采集到的监控视频进行实时的视频分析并展示分析结果，能够很好地降低延迟，并保证视频的流畅性。

[参考文献] (References)

[1] 闻辉,贾冬顺,严涛,车艳,林元模.智能视频监控系统的研究现状及展望[J].信息与电脑(理论版),2019(12):136-137.
[2] 郑健,冯瑞.基于Spark的实时视频分析系统[J].计算机系统应用,2017,26(12):51-57.
[3] Apache Software Fundation. Flink:Stateful Computations over Data Streams[OL]. https://flink.apache.org/
[4] 李广宏.vue.js前端应用技术分析[J].中国新通信,2019,21(20):115.
[5] Rafael Osso. Handbook of Emerging Communications Technologies: The Next Decade[M]. CRC Press, 1999.
[6] Real-Time Messaging Protocol[S]. America：Adobe Systems, 2012.
[7] 克雷格·沃斯. Spring Boot实战[M]. 丁雪丰. 北京：人民邮电出版社，2016.